



T.C. İSTANBUL KÜLTÜR ÜNİVERSİTESİ

COM5005 – WEB PROGRAMMING

LAB 09 – ASP.NET MVC5 CONTROLLER & VIEW

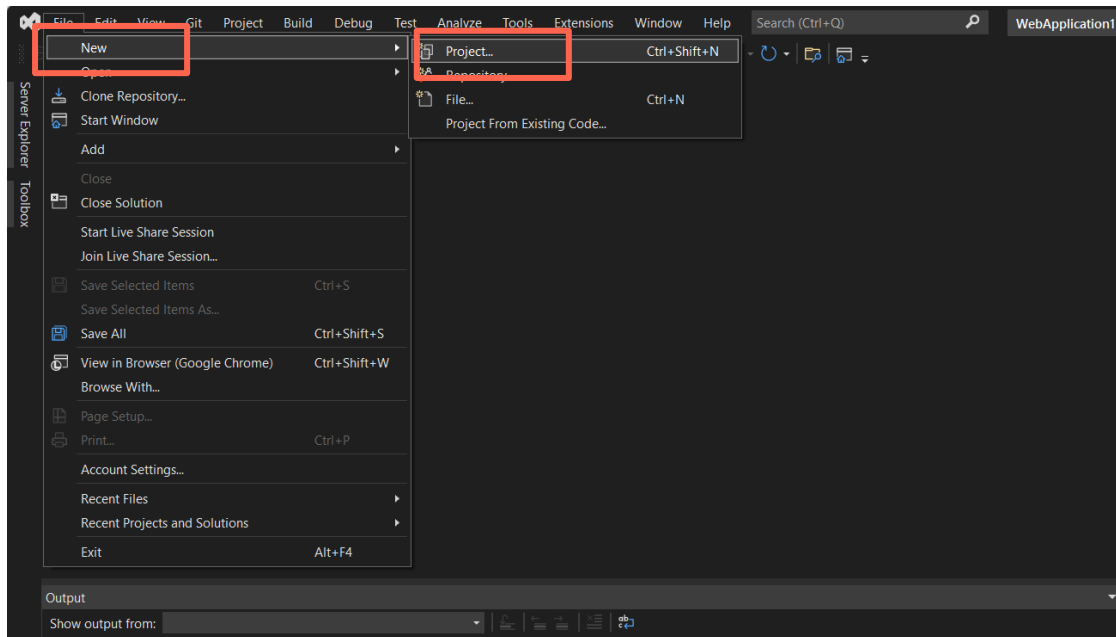
After completing this Lab, you will be able to

- All aspects of MVC5 Controller.
- How to change the default controller?
- ViewBag
- ViewData

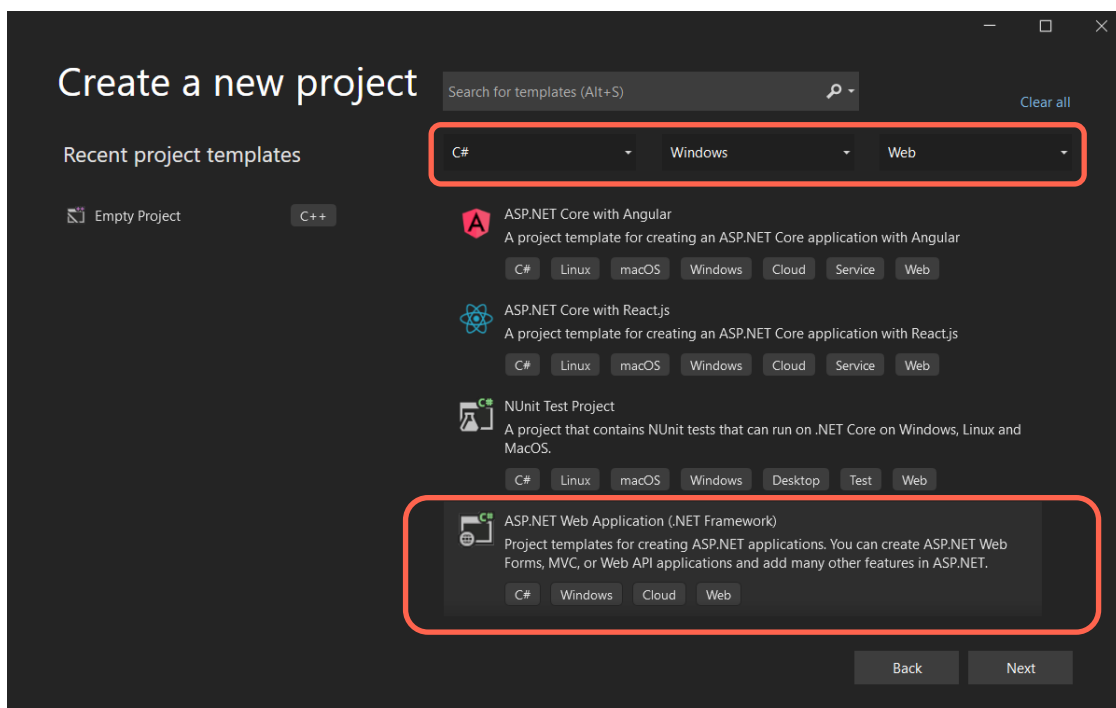
PROCEDURE 1 – All aspects of MVC5 Controller.

MVC controllers are responsible for responding to requests made against an ASP.NET MVC website. Each browser request is mapped to a particular controller. The ASP.NET MVC framework maps requested URLs to the classes that are referred to as controllers. Controller processes incoming requests, handle user input and interactions and executes appropriate business logic.

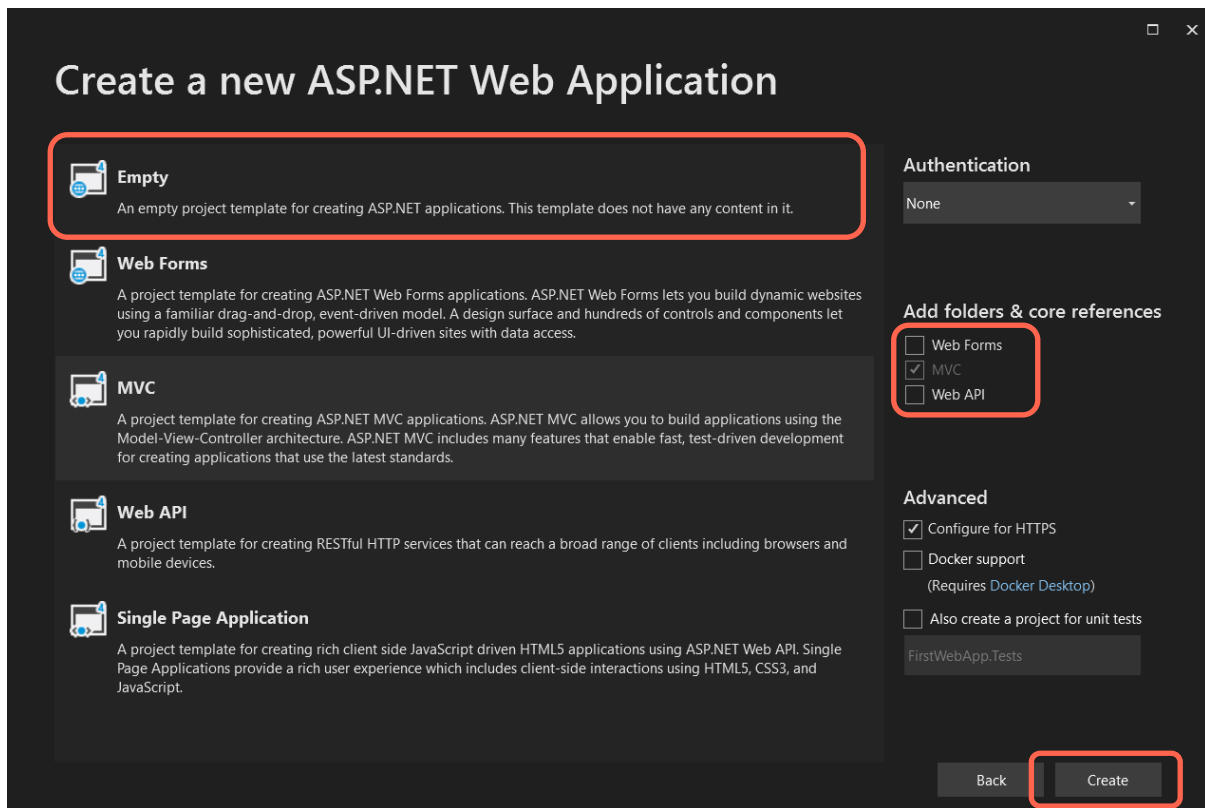
Step 1 – Create a Project.



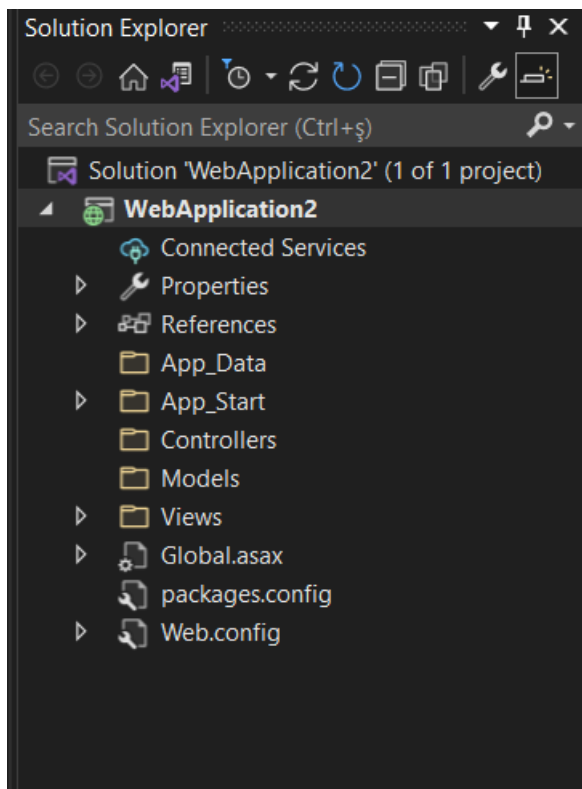
Step 2 – Select “ASP.NET Web Application” and provide a name.



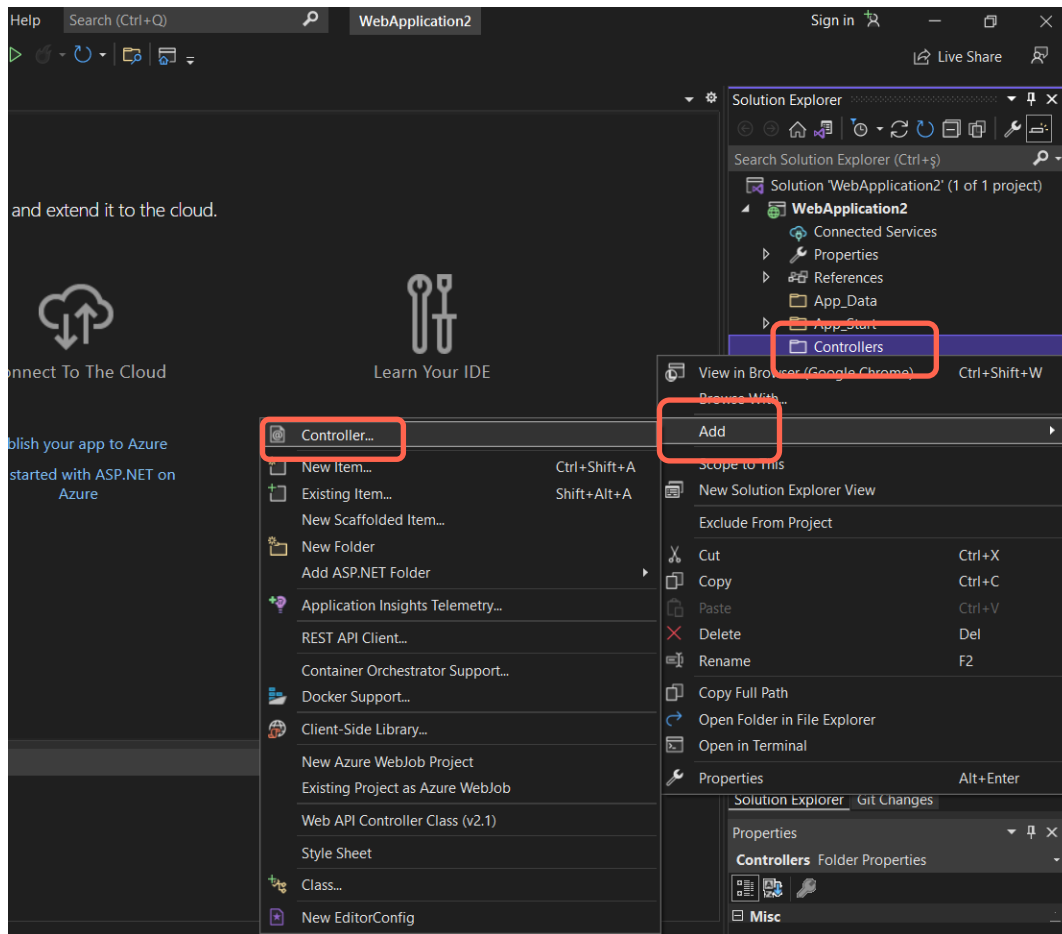
Step 3 – Select the “Empty” template and check the “MVC” checkbox from “New Web Application” window as follow.



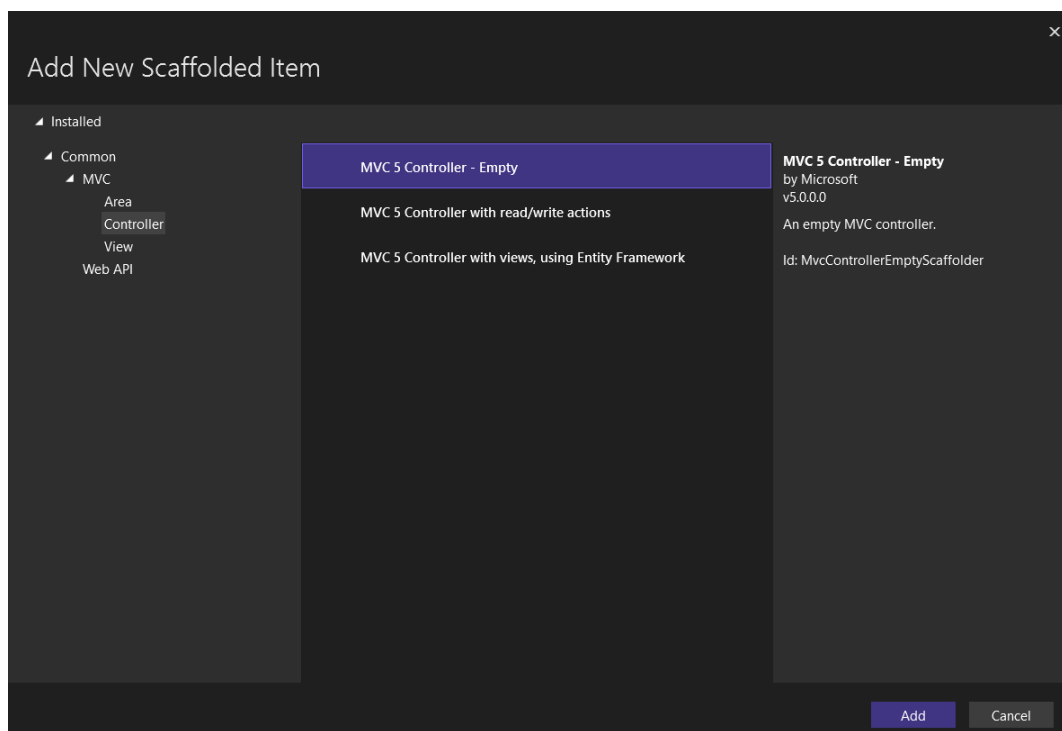
Step 4– Check the Solution Explorer of the Project and have a look at the files shown



Step 5 – Create a new controller.



Step 6 – Select “MVC 5 Empty Controller” from the window and click on “Add” button.

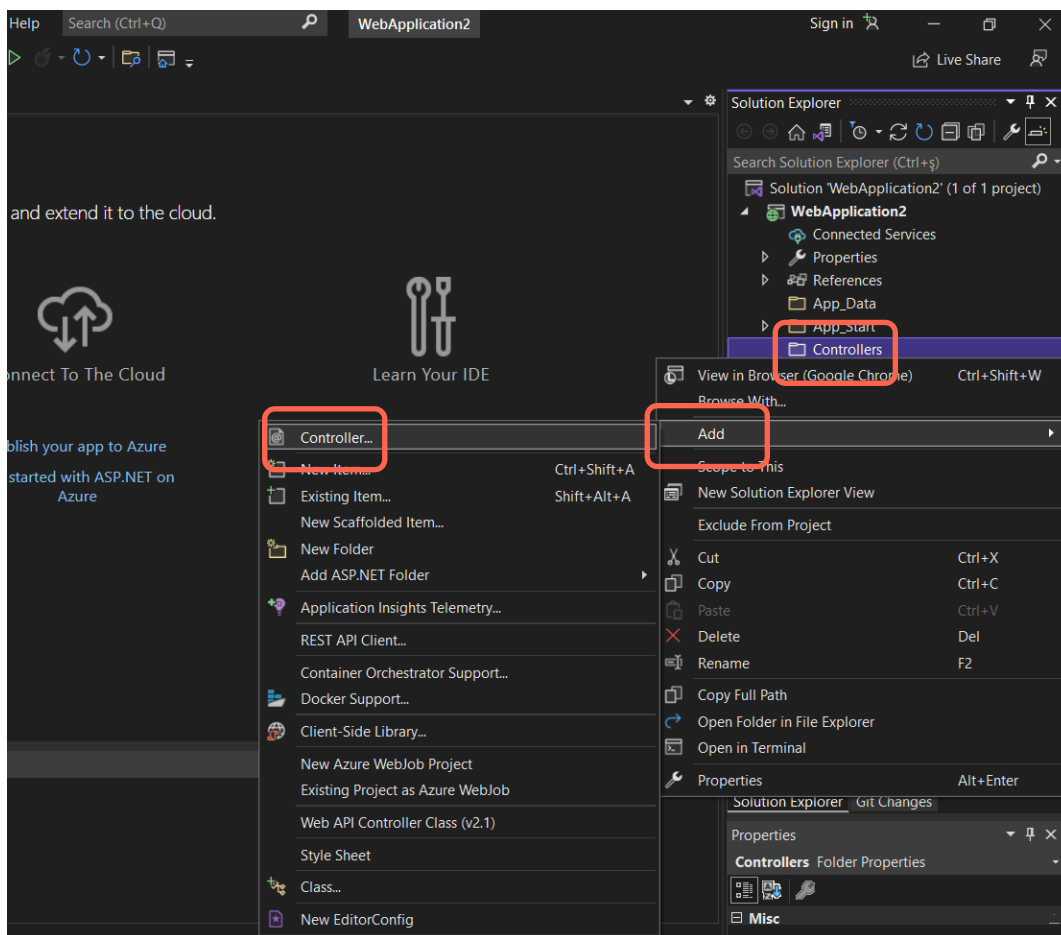


Step 7 – Provide a name like **“HomeController”** and Click on "Add" button.

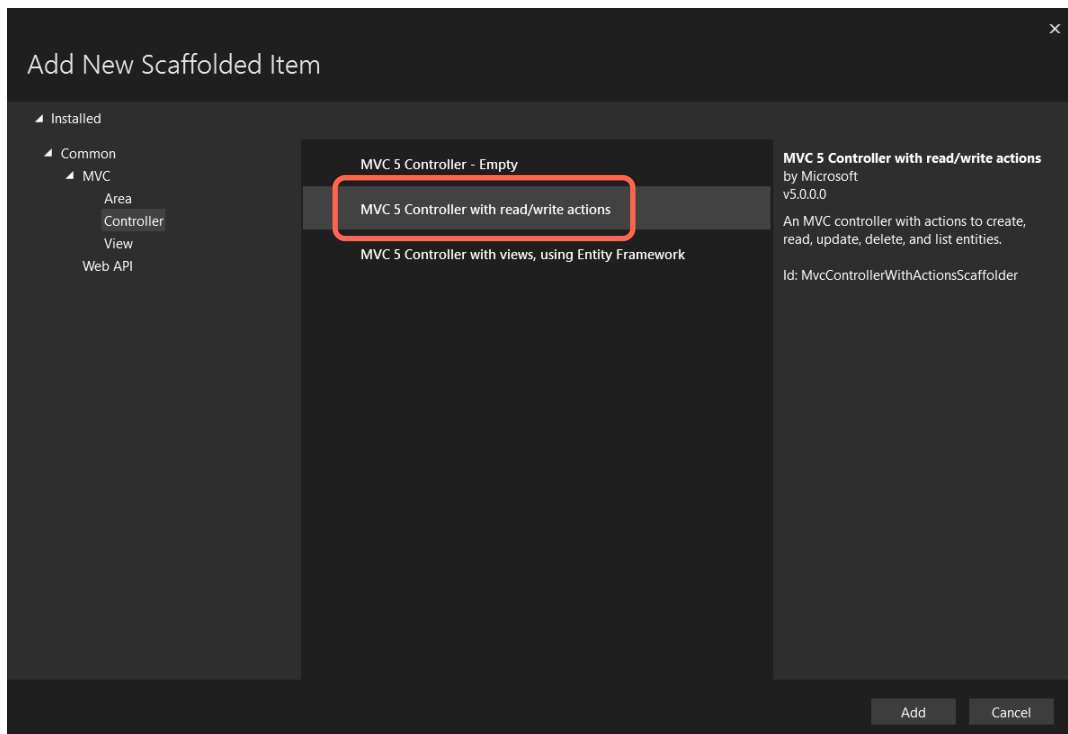


Step 8 – Go to solution explorer.

Right-click on **“Controller”** Folder >> Click on **Add** >> click on **“Controller”** as follow.



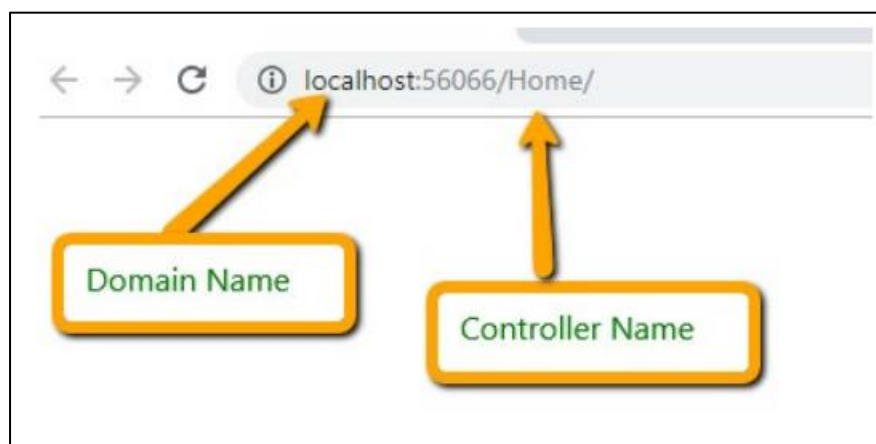
Step 9 – Select "MVC 5 Controller with read/write actions" from the window and click on "Add" button.



Step 10 – Provide a name like "ProductController" and Click on "Add" button as follow.



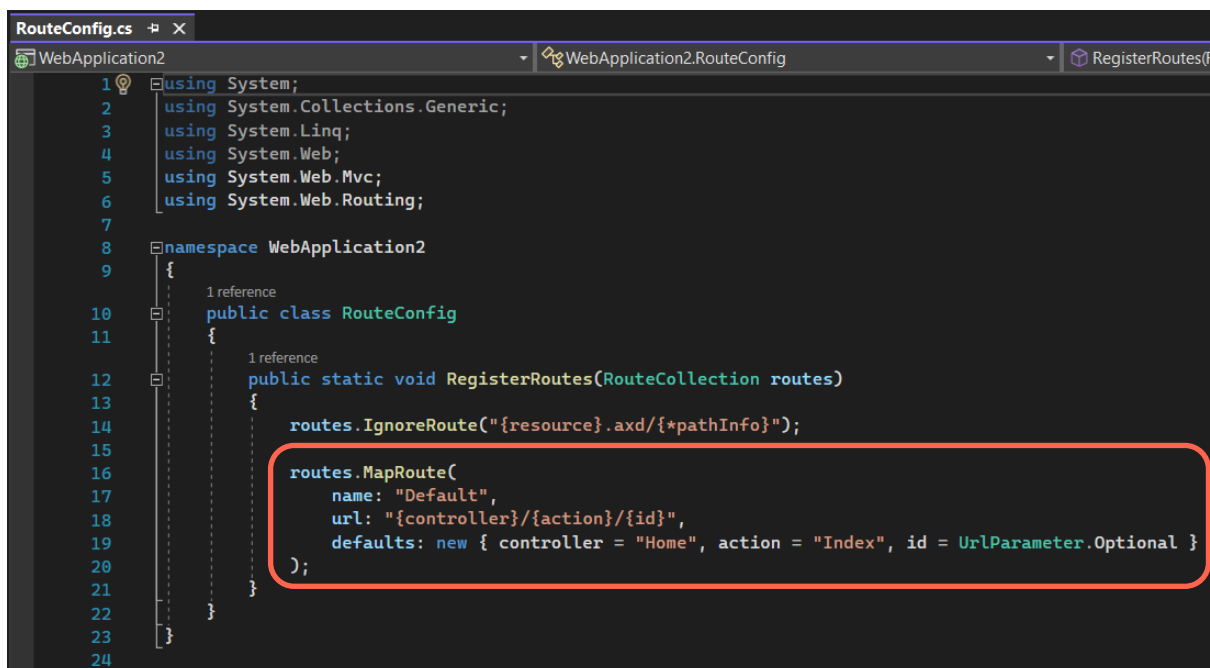
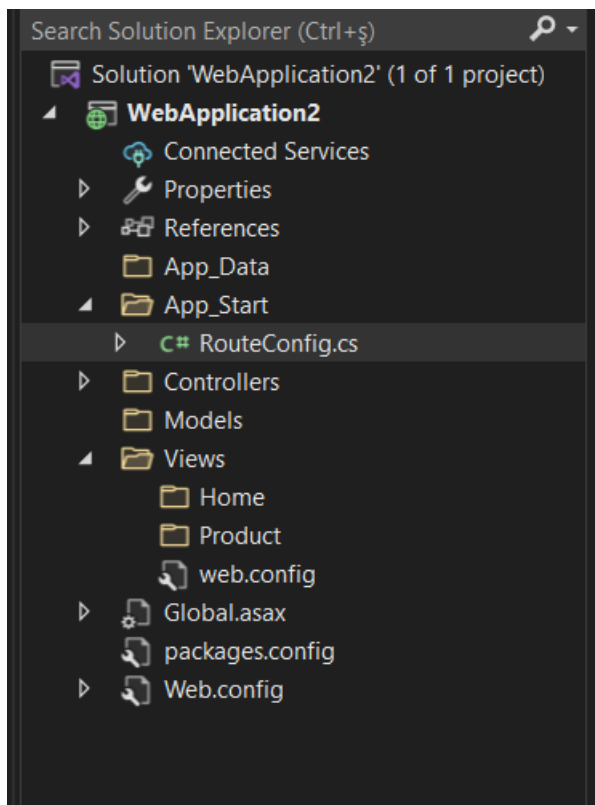
Step 11 – Open any browser and enter the URL like "DomainName/ControllerName" as follows.



PROCEDURE 2 – How to change the default controller?

Step 1 - Open the **Routeconfig.cs** file default file as follows.

Under the App_Start folder >> **RouteConfig.cs**



Change the controller name as "**Product**" controller instead of the "Home" controller inside the RouteConfig.cs file as follow.

```
RouteConfig.cs* X
WebApplication2
WebApplication2.RouteConfig
RegisterRoutes

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Web;
5 using System.Web.Mvc;
6 using System.Web.Routing;
7
8 namespace WebApplication2
9 {
10     1 reference
11     public class RouteConfig
12     {
13         1 reference
14         public static void RegisterRoutes(RouteCollection routes)
15         {
16             routes.IgnoreRoute("{resource}.axd/{*pathInfo}");
17
18             routes.MapRoute(
19                 name: "Default",
20                 url: "{controller}/{action}/{id}",
21                 defaults: new { controller = "Product", action = "Index", id = UrlParameter.Optional }
22             );
23         }
24     }
25 }
```

PROCEDURE 3 – All aspects of MVC5 View.

A view is used to display data using the model class object. The Views folder contains all the view files in the ASP.NET MVC application. A controller can have one or more action methods, and each action method can return a different view. In short, a controller can render one or more views.

! So, for easy maintenance, the MVC framework requires a separate sub-folder for each controller with the same name as a controller, under the Views folder.

Step 1 - Create New Controller Actions. Write new Login controller method in HomeController. In the Search Explorer window, the Login() action controller will be created automatically under the HomeController.

```
WebApplication2
WebApplication2.Controllers.HomeController
Search Solution Explorer (Ctrl+Shift)

2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Web;
5 using System.Web.Mvc;
6
7 namespace WebApplication2.Controllers
8 {
9     0 references
10     public class HomeController : Controller
11     {
12         // GET: Home
13         0 references
14         public ActionResult Index()
15         {
16             return View();
17         }
18
19         0 references
20         public ActionResult Login()
21         {
22             return View();
23         }
24     }
25 }
```

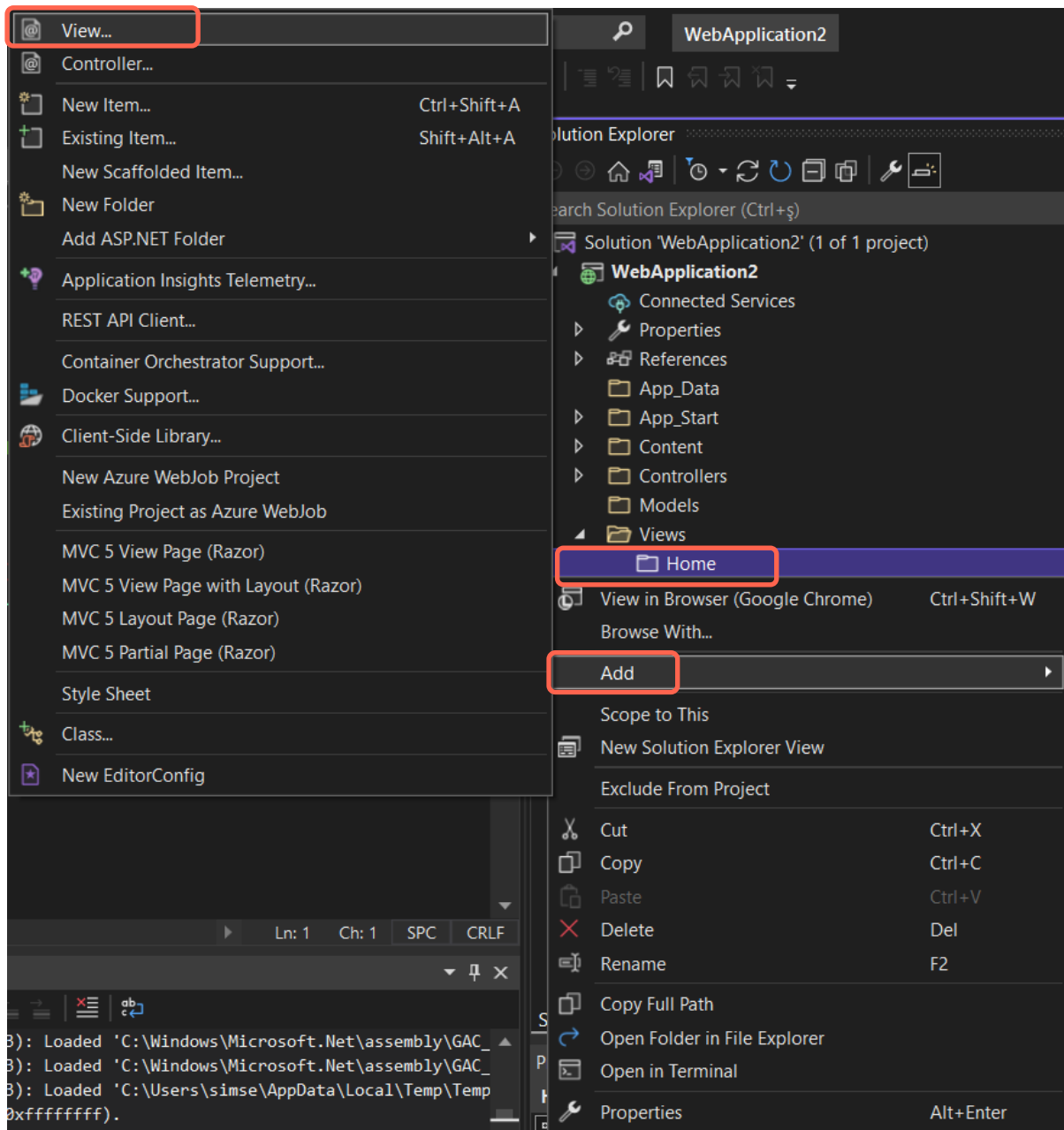
Search Solution Explorer (Ctrl+Shift)

- Solution 'WebApplication2' (1 of 1 project)
- WebApplication2
 - Connected Services
 - Properties
 - References
 - App_Data
 - App_Start
 - RouteConfig.cs
 - Controllers
 - HomeController.cs
 - Index() : ActionResult
 - Login() : ActionResult
 - ProductController.cs
 - Models
 - Views
 - Home
 - Product
 - web.config
- Global.asax
- packages.config
- Web.config

Step 2 - Create/add **View** for the created action controllers.

Under the View folder >> right click to Home folder >> Add >> click View..

Step 3 - Enter the new view name the same as the controller name. (eg. Login)



Step 4 - Adding new views (Login.cshtml) to the layout: Open the **_Layout.cshtml**.

```
<body>
  <div class="navbar navbar-inverse navbar-fixed-top">
    <div class="container">
      <div class="navbar-header">
        <button type="button" class="navbar-toggle" data-toggle="collapse" data-target=".navbar-collapse">
          <span class="icon-bar"></span>
          <span class="icon-bar"></span>
          <span class="icon-bar"></span>
        </button>
        <@Html.ActionLink("Application name", "Index", "Home", new { area = "" }, new { @class = "navbar-brand" })>
      </div>
      <div class="navbar-collapse collapse">
        <ul class="nav navbar-nav">
          <li><@Html.ActionLink("Login", "Login", "Home")></li>
        </ul>
      </div>
    </div>
  </div>

  <div class="container body-content">
    @RenderBody()
    <hr />
    <footer>
      <p>&copy; @DateTime.Now.Year - My ASP.NET Application</p>
    </footer>
  </div>
```

!_Run the Project and check if the new added views are showing and working correctly

Application name

- [Login](#)

Change name of Link. Go Layout.cshtml.

New View * Login Web Page

© 2022 - My ASP.NET Application

PROCEDURE 4 – ViewBag.

The ViewBag in ASP.NET MVC is used to transfer temporary data (which is not included in the model) from the controller to the view.

Controllers >> HomeController.cs :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;

namespace WebApplication2.Controllers
{
    0 references
    public class HomeController : Controller
    {
        // GET: Home
        0 references
        public ActionResult Index()
        {
            return View();
        }

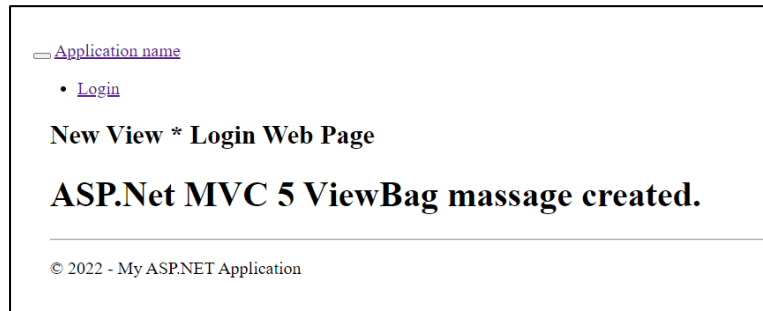
        0 references
        public ActionResult Login()
        {
            ViewBag.message = "ASP.Net MVC 5 ViewBag message created.";

            return View();
        }
    }
}
```

View >> Home >> Login.cshtml

```
1
2  @{
3      ViewBag.Title = "Login";
4  }
5
6  <h2>New View * Login Web Page</h2>
7
8  <h1>@ViewBag.message</h1>
9
```

**** Execute the code and review the results.**



PROCEDURE 5 – ViewData.

ViewData is similar to ViewBag, which transfers data from Controller to View. ViewData is of Dictionary type, whereas ViewBag is of dynamic type. However, both store data in the same dictionary internally.

```
0 references
public ActionResult Login()
{
    ViewBag.message = "ASP.Net MVC 5 ViewBag message created.";
    ViewData["message"] = "ASP.Net MVC 5 ViewBag message created.";

    return View();
}
```

```
@{
    ViewBag.Title = "Login";
}

<h2>New View * Login Web Page</h2>

<h1>@ViewBag.message</h1>
<h1>@ViewData["message"]</h1>
```

** Execute the code and review the results.

